Syntaris Build Plan

Multi-Agent Research Assistant

Full spec, setup guide, and 6 additional portfolio project ideas

Chris Schmidt | ML Engineer | JHU AI Engineering

Contents

1. What This Document Is

This document serves as your complete build brief for developing a Multi-Agent Research Assistant using the Syntaris scaffold in Claude Code (VS Code extension). It contains everything you need to go from a fresh repo to a running Phase 1 checkpoint.

Syntaris operates on a gate model: Claude Code cannot advance to the next phase until you explicitly approve the current phase output. This document is structured to align with that model, giving you the context to make each approval confidently.

Build Type: Production

This is a portfolio-grade production build, not a prototype.
Final version target: v1.0 Production Live.
Estimated total hours across all gates: 22-39h (spread across sessions).
Syntaris will calibrate hour estimates gate-by-gate as you build.

2. Project Overview

What It Is

A multi-agent AI research assistant that accepts a natural-language research question, decomposes it into sub-questions, retrieves relevant document chunks from an indexed knowledge base, critiques and filters those chunks, and synthesizes a cited answer - all streamed to a clean web interface.

Unlike your existing RAG journal summarizer (which is a single-model pipeline), this app exposes the multi-agent orchestration layer explicitly: each agent node is named, observable in the UI timeline, and independently testable. This is the architecture gap your portfolio currently has.

Why It Fits Your Portfolio

- Fills the agentic/multi-agent AI gap identified in your portfolio review
- Moves LLM usage from black-box API calls to an orchestrated LangGraph DAG
- RAGAS evals are a first-class gate deliverable - evals culture is visible in the artifact trail
- LangSmith tracing shows professional observability practice
- Supabase + FastAPI is production-grade, not a Jupyter notebook
- Streaming UI demonstrates full-stack ML engineering, not just modeling

What a Visitor Sees

1. Paste a research question into a chat panel
2. Watch a live agent-status timeline: Planner > Retriever > Critic > Synthesizer
3. Read a streamed answer with inline numbered citations
4. Click any citation to see the source chunk in a side panel
5. See RAGAS eval scores for the response (faithfulness, answer relevancy) in a footer badge

3. Full Tech Stack

This is the nextjs-fastapi-supabase recipe with LangGraph added to the FastAPI backend. Every component is in the Syntaris recipe; nothing requires custom configuration beyond what the install script provides.

Layer	Technology	Role in App
Frontend	Next.js 14+ (App Router)	Chat UI, source panel, streaming SSE consumer, auth pages
Backend	FastAPI (Python)	Agent orchestration endpoint, streaming response, RAGAS eval routes
Agents	LangGraph	Multi-agent DAG: Planner, Retriever, Critic, Synthesizer nodes
Embeddings	text-embedding-3-small	Document chunks -> vector store for retrieval
Vector Store	pgvector (Supabase)	Stores document embeddings alongside relational data
LLM	Claude claude-sonnet-4-20250514 via API	Powers all agent nodes; swap per-node as needed
Database	Supabase (PostgreSQL)	User sessions, research sessions, source metadata, eval logs
Auth	Supabase Auth	Email/OAuth, RLS on all tables
Evals	RAGAS + LangSmith	Faithfulness, answer relevancy, context precision; traces logged
Infra	Docker Compose	Local dev; single-command startup
CI/CD	GitHub Actions	Lint, test, deploy on merge to main

Supabase Tables (MVP)

users - Supabase Auth managed
research_sessions - id, user_id, query, created_at
sources - id, session_id, content, metadata, embedding (vector)
eval_logs - id, session_id, faithfulness, answer_relevancy, context_precision

4. Agent Architecture

The backend is a LangGraph StateGraph with five nodes wired in sequence. The Evaluator node runs asynchronously after the response is delivered so it does not affect latency.

Agent	LangGraph Node	Responsibility	Model / Tool Use
Planner	plan_node	Parses user query; decomposes into sub-questions; sets retrieval strategy	Sonnet - structured output (Pydantic)
Retriever	retrieve_node	Semantic search against pgvector; re-ranks chunks by relevance score	Embedding API + pgvector cosine sim
Critic	critique_node	Scores retrieved chunks for relevance; filters noise; flags gaps	Sonnet - yes/no classification per chunk
Synthesizer	synthesize_node	Generates final answer with inline citations; streams response to client	Sonnet - streaming, citation formatting
Evaluator	eval_node (async)	Runs RAGAS metrics post-response; logs to LangSmith; writes to Supabase	RAGAS library - no LLM call in hot path

LangGraph State Schema (Pydantic)

The shared state object passed between nodes looks like this:

```
class ResearchState(BaseModel):
    query: str
    sub_questions: list[str] = []
    retrieved_chunks: list[Chunk] = []
    filtered_chunks: list[Chunk] = []
    answer: str = ""
    citations: list[Citation] = []
    eval_scores: EvalScores | None = None
```

Design Decision: Why LangGraph over raw function calls?

LangGraph's StateGraph makes the agent DAG inspectable - you can add breakpoints, replay nodes, and trace exactly what each agent received and produced.

This directly addresses the "black-box API calls" gap. A portfolio reviewer can see the graph definition and understand the architecture immediately.

LangSmith integrates natively with LangGraph for zero-config tracing.

5. Syntaris Gate Roadmap

This is the full v0.0 - v1.0 roadmap you will paste into Syntaris at SCOPE CONFIRMED. The gate structure maps to the Syntaris v11.4 five-phase model.

Gate	Deliverables	You Approve When...	Est. Hours
SCOPE CONFIRMED	CONTRACT.md, SPEC.md, agent map, full version roadmap v0.0-v1.0	Domain, agent roles, data flows, API keys scoped; no ambiguity remains	**2-3h**
MOCKUPS APPROVED	Wireframes (Excalidraw or screenshots), DESIGN_SYSTEM.md, FRONTEND_SPEC.md	Chat panel, source sidebar, citation renderer, agent-status timeline look right	**3-5h**
FRONTEND APPROVED	Working Next.js UI, component tests > 0, screenshots passing	UI renders, streaming works, no console errors, Playwright smoke tests green	**8-15h**
TESTS APPROVED	RAGAS eval suite, LangSmith traces wired, test plan doc	Faithfulness >= 0.75, answer relevancy >= 0.70, evals are reproducible	**5-8h**
GO	Supabase schema migrated, Docker Compose, README, deployed URL	App is live, CI passes, README covers setup from clone to run	**4-8h**

Syntaris Gate Mechanics

Each gate has an approval token you type (e.g., "SCOPE CONFIRMED"). Without it, hooks block Claude Code from advancing.

At each gate close, Syntaris writes a snapshot to .blueprint/snapshots/<version>/ and tags the git commit.

If a gate goes wrong, /rollback restores to the last closed gate - code and memory files.

Calibration: actual vs. estimated hours are logged to MEMORY_CORRECTIONS.md; future estimates adjust automatically.

6. Step-by-Step Setup Guide

Prerequisites (do these before opening VS Code)

You need all of these installed and working before running the Syntaris installer:

6. **Node.js 18+:** Required by Claude Code and the Syntaris install script. Run: node --version
7. **Git:** Required by Syntaris hooks for gate tagging and rollback. Run: git --version
8. **Python 3.11+:** Required for the FastAPI backend. Run: python3 --version
9. **jq:** Required by Syntaris bash hooks. macOS: brew install jq | Ubuntu: apt install jq
10. **Claude Code VS Code extension:** Install from the VS Code marketplace. Sign in with your Anthropic account.
11. **Anthropic API key:** Set as ANTHROPIC_API_KEY in your shell profile or .env file.
12. **Supabase account:** Free tier works. Create a new project at supabase.com. Note your project URL and anon key.
13. **LangSmith account:** Free tier works. Get your API key at smith.langchain.com.

Phase A: Install Syntaris

Run these commands in your terminal. The installer puts skills, hooks, and agents into ~/.claude/ where Claude Code can find them.

14. **Clone your Syntaris fork:**

```
git clone https://github.com/PCSchmidt/Syntaris.git ~/Syntaris
cd ~/Syntaris
```

15. **Configure your personal overlay:**

Copy the template and fill in your details:

```
cp personal-overlay/owner-config.template.md personal-overlay/owner-config.md
```

Open the file and replace placeholder values with your name, GitHub username, etc.

16. **Run the installer:**

```
bash install.sh
```

The installer will ask for confirmation before clobbering any existing install. Type yes to proceed.

17. **Verify the install:**

```
bash verify.sh
```

You want to see: 84+ passes, 0 failures. If anything fails, run collect-diagnostics.sh and check TROUBLESHOOTING.md.

Phase B: Create the Project Repo

18. **Create a new GitHub repo:** Name it something like multi-agent-research-assistant. Initialize with a README. Make it public (portfolio visibility).
19. **Clone the repo locally:**

```
git clone https://github.com/PCSchmidt/multi-agent-research-assistant.git
cd multi-agent-research-assistant
```

20. **Open in VS Code:**

```
code .
```

21. **Confirm Claude Code sees Syntaris:** Open the Claude Code panel. Type /start. If Syntaris is installed correctly, you will see the Blueprint greeting and the Phase 1 interrogation prompt.

Phase C: SCOPE CONFIRMED (Gate 0)

This is where Syntaris generates your full roadmap. You do NOT write code yet - you answer Claude Code's questions. See Section 7 for the exact dump to paste.

22. **Type /start in Claude Code.** Claude Code will ask you the build-type question (Production / Internal / Exploratory). Answer: Production.
23. **Paste the dump from Section 7.** This gives Claude Code everything it needs to generate CONTRACT.md, SPEC.md, and the full version roadmap without asking clarifying questions one at a time.
24. **Review the generated documents.** Claude Code will produce CONTRACT.md and VERSION_ROADMAP.md. Read both. If anything is wrong, say so now - scope changes after this gate require re-approval.
25. **Type the approval token.** When you are satisfied, type exactly: SCOPE CONFIRMED. The hooks will unlock Phase 2.

Phase D: Work Through the Gates

After SCOPE CONFIRMED, follow the Syntaris gate flow. At each gate:

- Read what Claude Code proposes before it writes anything substantial
- Use /critical-thinker if a decision feels uncertain
- Use /costs before any gate that touches infrastructure
- Use /security before the FRONTEND APPROVED gate
- Use /rollback if a gate goes wrong - it restores to the last clean state

> **Session Management Tip**
>
> Syntaris's context-check hook will warn you at 80 turns and hard-stop at 120.
>
> At the start of each new session, type /start - it re-injects project memory from the foundation files.
>
> If you notice Claude Code forgetting context mid-session, /health will show you the current context window state.

7. Scope Confirmation Dump

Paste this entire block into Claude Code after typing /start and selecting Production as your build type. It is designed to answer all of Syntaris's standard clarifying questions upfront, minimizing back-and-forth.

How to Use This Dump

1. Open Claude Code in VS Code with your new project repo open.
2. Type /start in the Claude Code chat panel.
3. When asked for build type, respond: Production.
4. When Claude Code asks for your project dump, paste the entire block below.
5. Claude Code will generate CONTRACT.md, SPEC.md, DECISIONS.md, and VERSION_ROADMAP.md.
6. Review each file, request changes if needed, then type: SCOPE CONFIRMED.

--- BEGIN SCOPE DUMP ---

PROJECT NAME: multi-agent-research-assistant

BUILD TYPE: Production

FINAL VERSION: v1.0 Production Live

What I am building

A multi-agent AI research assistant. The user pastes a research question. A LangGraph agent pipeline decomposes the question, retrieves relevant document chunks from a pgvector store, critiques and filters those chunks, and streams a cited answer back to the user through a Next.js frontend.

Why I am building it

Portfolio project. I am an ML engineer with five years of experience building toward a stronger agentic AI portfolio. This addresses three specific gaps: no visible multi-agent work, LLM usage limited to black-box API calls, and no evals culture. The app needs to be production-quality, deployed, and visible on GitHub.

Target users

Primary: technical reviewers (hiring managers, senior engineers) evaluating my portfolio. Secondary: myself as a research tool. The app should feel like something I would actually use, not a demo with fake data.

Stack decisions (final, not negotiable)

- Frontend: Next.js 14 App Router, TypeScript, Tailwind
- Backend: FastAPI (Python 3.11), LangGraph for agent orchestration
- LLM: Claude claude-sonnet-4-20250514 via Anthropic API

- Embeddings: text-embedding-3-small (OpenAI) or Claude when available
- Vector store: pgvector extension on Supabase
- Database / Auth: Supabase
- Evals: RAGAS + LangSmith
- Infra: Docker Compose for local dev, GitHub Actions for CI

Agent architecture (four active nodes + one async eval node)

- Planner: decomposes query into sub-questions, sets retrieval strategy
- Retriever: semantic search against pgvector, returns top-k chunks
- Critic: scores each chunk for relevance, filters noise
- Synthesizer: generates streamed answer with inline citations
- Evaluator: runs RAGAS post-response, logs to LangSmith and Supabase (async, not in hot path)

UI requirements

- Chat panel: single text input, streaming response with inline citations (numbered superscripts)
- Agent timeline: real-time status indicators showing which node is active
- Source panel: clicking a citation opens the chunk in a side panel
- Eval badge: shows faithfulness and answer relevancy scores after response completes
- Auth: Supabase email auth, sessions persisted

RAGAS eval targets (these are the TESTS APPROVED gate thresholds)

- Faithfulness >= 0.75
- Answer relevancy >= 0.70
- Context precision >= 0.65
- Evals must be reproducible (seeded test set, not ad-hoc)

What I am NOT building (out of scope for v1.0)

- Web search / live internet retrieval - knowledge base is document-indexed only
- Multi-user collaboration - single user per session
- Document upload UI - seed the vector store via a CLI script
- Fine-tuning - inference only
- Mobile app

API keys I have

- ANTHROPIC_API_KEY - set in environment
- OPENAI_API_KEY - for embeddings (set in environment)
- LANGCHAIN_API_KEY - LangSmith (set in environment)
- SUPABASE_URL and SUPABASE_ANON_KEY - from Supabase project settings

--- END SCOPE DUMP ---

8. Six Additional Portfolio Project Ideas

Each of these is matched to your background, portfolio gaps, and personal interests. They are ordered from highest portfolio ROI to most personally useful.

Project	Stack / Recipe	Portfolio Signal	Personal Angle
RAGAS Eval Dashboard	nextjs-fastapi-supabase	Evals culture, LangSmith, RAGAS metrics visible	Built on your existing RAG journal summarizer - zero greenfield risk
LoRA Experiment Tracker	nextjs-fastapi (SQLite)	Fine-tuning competency, MLOps discipline, run comparison	Direct support for your JHU QLoRA project - tooling you will actually use
Construction PM Tool	nextjs-fastapi-supabase	Full-stack product sense, domain modeling, auth + RLS	Self-GC house build near Franklin, NC - a real working tool not a demo
Agentic Stock Screener	nextjs-fastapi-supabase + LangGraph	Agentic reasoning, structured output, financial domain	Directly aligns with your investing interest; ties to your ML engineering skills
DBN/PGM Explainer App	python-cli + React front	Graduate-level ML knowledge, teaching ability, visualization	Builds on your Bayesian networks and RL coursework from JHU - you own this material
JHU Coursework Portfolio Site	Next.js SSG	Breadth signal, academic rigor, presentation quality	Replaces or complements pcschmidt.github.io with richer project case studies

Project 1: RAGAS Evaluation Dashboard

What it is: A FastAPI backend that runs RAGAS metrics against your existing RAG journal summarizer on demand, with a Next.js dashboard visualizing faithfulness, answer relevancy, and context precision trends over time.

Why it matters: Your journal summarizer is deployed but the evals are invisible. This makes them visible - a chart of metric trends over time is more impressive than a README line saying "evaluated with RAGAS." It also demonstrates eval engineering as a discipline, not an afterthought.

Syntaris recipe: nextjs-fastapi-supabase. Lean build - most complexity is in the eval pipeline, not the UI.

Estimated gates: 3 gates total (SCOPE CONFIRMED, FRONTEND APPROVED, GO). No TESTS APPROVED gate since testing IS the product.

RAGAS metrics to expose: Faithfulness, Answer Relevancy, Context Precision, Context Recall. Store per-query history in Supabase.

Project 2: LoRA/QLoRA Experiment Tracker

What it is: A lightweight MLflow-inspired experiment tracker tailored for fine-tuning runs. Logs hyperparameters, training loss curves, and eval metrics. Compares runs side-by-side.

Why it matters: The fine-tuning project on your roadmap needs tooling. Building that tooling yourself is itself a portfolio signal - it shows MLOps discipline and that you think about the full training workflow, not just the model.

Syntaris recipe: python-cli for the logging SDK, then promote to nextjs-fastapi with SQLite (not Supabase - keep it portable). A researcher should be able to run this locally with zero cloud dependencies.

Key differentiator: Export a run comparison to a shareable HTML report. This makes the fine-tuning portfolio entry self-contained - the report is the evidence.

Project 3: Construction Project Management Tool

What it is: A project management app for self-GC residential construction: subcontractor scheduling with phase gates, permit tracking with status and expiry dates, materials cost logging vs. budget, and document storage (plans, bids, contracts).

Why it matters: You are building a 3-4 bedroom house near Franklin, NC and self-GC'ing it. This is a real production tool with a real user - you. Interviewers respond well to apps that solve a real personal problem. The domain modeling (phases, dependencies, budget roll-ups) is genuinely interesting.

Syntaris recipe: nextjs-fastapi-supabase. The Syntaris gate model is almost a metaphor for construction phases.

Portfolio angle: The README case study writes itself: "Built to manage my own $X house build. Currently tracking Y subcontractors across Z active permits."

Project 4: Agentic Stock Screener

What it is: An agentic pipeline that accepts a natural-language screening query ("profitable small-caps with growing FCF and low debt"), decomposes it into structured financial filters, fetches data from a financial API, scores candidates, and returns a ranked list with reasoning.

Why it matters: Combines your investing interest with agentic AI. The interesting technical challenge is the query-to-structured-filter decomposition - it requires a Planner agent that produces valid API parameters from freeform text. This is a meaningful NLU problem, not a toy.

Syntaris recipe: nextjs-fastapi-supabase + LangGraph. Use yfinance or Polygon.io for data (both have free tiers).

RAGAS angle: Eval the Planner's filter accuracy against a test set of query-to-filter pairs. This adds evals culture to a domain outside NLP, showing breadth.

Project 5: DBN/PGM Interactive Explainer

What it is: An interactive web app that lets users build small Bayesian networks and dynamic Bayesian networks visually, then runs inference (variable elimination, likelihood weighting) and shows the step-by-step computation. Built for teaching.

Why it matters: You have deep coverage of this material from JHU coursework. Building a teaching tool for a topic you own cold is a strong signal - it shows you can explain, not just implement. This addresses the "no visible graduate-level ML knowledge" gap without being another notebook.

Syntaris recipe: python-cli + React front (bring-your-own template). Backend is pgmpy for inference. Frontend uses React Flow or D3 for the graph editor.

Portfolio angle: Link it from your portfolio as "Try variable elimination on a network you build in 30 seconds." Interactive demos get more engagement than static READMEs.

Project 6: JHU Coursework Portfolio Site

What it is: A replacement or companion to pcschmidt.github.io that presents your JHU coursework and portfolio projects as structured case studies: problem statement, technical approach, results, code link, and one visual per project.

Why it matters: Your current portfolio has identified gaps in presentation. A dedicated case-study format forces you to articulate the "so what" of each project, which is what interviewers actually read. The site itself is a portfolio signal - a well-designed static site shows frontend competence.

Syntaris recipe: Next.js SSG (bring-your-own template). No backend needed. Deploy to Vercel or keep on GitHub Pages.

Key feature: A skills matrix that maps each project to the competencies it demonstrates. Makes it easy for a reviewer to cross-reference "do you have LangGraph experience?" to a specific project entry.

9. Decision Checklist

Work through this before you type /start in Claude Code. Having clear answers to these questions will make the SCOPE CONFIRMED gate go cleanly.

Before you open Claude Code

[ ] verify.sh returns 84+ passes, 0 failures
[ ] ANTHROPIC_API_KEY is set in your shell profile
[ ] OPENAI_API_KEY is set (for embeddings)
[ ] LANGCHAIN_API_KEY is set (LangSmith)
[ ] Supabase project created; URL and anon key noted
[ ] personal-overlay/owner-config.md filled in from template
[ ] New GitHub repo created and cloned locally
[ ] Repo is open in VS Code with Claude Code extension active

Questions Claude Code will ask at SCOPE CONFIRMED

Build type: Production / Internal / Exploratory -> Answer: Production
What problem does this solve? -> Covered in the scope dump.
Who are the users? -> Covered in the scope dump.
What does v1.0 look like? -> Covered in the scope dump.
Any cost or timeline constraints? -> State your weekly hours budget here.
Any hard technical constraints? -> Stack is fixed (see dump). Claude Code should not deviate.

Ready to build. Type /start.